

Rapport de stage

Adèle Dejoie

9 décembre 2025

Table des matières

1	Introduction	3
2	Premier algorithme	3
2.1	Grammaire algébrique	3
2.2	Implémentation dans un algorithme	4
2.2.1	Plan de conception	4
2.2.2	Grammaire utilisée	4
2.2.3	Utilisation de problog	5
2.3	Résultats obtenus	7
2.4	Problèmes	7
3	Deuxième algorithme	8
3.1	Graphe bayésien	8
3.1.1	Définition	8
3.1.2	Hypothèses	9
3.1.3	Utilisation des graphes bayésiens	10
3.1.4	Limites	12
3.2	Implémentation dans un algorithme	13
3.3	Résultats	15
4	Améliorations	16
4.1	Complexification des prémices	16
4.2	Ajout du do-calculus	21
5	Entraînement	23
5.1	Quelques notions	23
5.2	Algorithme d'entraînement	24
5.3	Résultats	26
6	Sources	26
7	Annexe	27

1 Introduction

J'ai réalisé mon stage sous la direction de Damien Siléo à l'INRIA Lille. Le but de mon stage était de générer des problèmes probabilistes et les réponses qui vont avec dans le but d'entraîner un réseau de neurone.

Un problème probabiliste est une énoncé composée de prémices définissant la situation, les événements, leur probabilité et les liens causaux qui les unissent. L'énoncé se conclut par une question portant sur le problème.

Durant mon stage j'ai principalement travaillé sur deux algorithmes qui seront présentés en partie II, III et IV. En partie V nous verrons un exemple simple d'utilisation que j'ai réalisé à la toute fin de mon stage.

2 Premier algorithme

Le but de cette partie est de présenter le premier algorithme sur lequel j'ai travaillé pour générer mon jeu de données. L'algorithme fait appel à la notion de grammaire algébrique qui est introduite dans la première sous partie.

2.1 Grammaire algébrique

Définition 2.1 (Alphabet)

Un alphabet est un ensemble de lettre.

Définition 2.2 (Grammaire algébrique)

Une grammaire algébrique est un quadruplet $G = (\Sigma, V, S, P)$ où :

- Σ est un alphabet fini, l'alphabet des terminaux. On utilise usuellement des minuscules pour les terminaux.
- V est un alphabet fini disjoint de Σ , l'alphabet des variables. On utilise habituellement des majuscules pour les variables.
- $S \in V$ est appelé axiome ou start.
- $P \subset V \times (\Sigma \cup V)^*$, l'ensemble des règles de production / dérivation.

Exemple 2.1. Soit $G_1 = (\{a\}, \{S\}, S, \{(S, aS), (S, \epsilon)\})$. ϵ représente le mot vide.

Définition 2.3 (Dérivation)

Soit $u, v \in (\Sigma \cup V)^$.*

On dit que u se dérive en v s'il existe $X \in V$, $u_1, u_2, v_1, v_2, m \in (\Sigma \cup V)^$ tels que :*

- $(X, m) \in P$
- $u = u_1 X u_2$
- $v = v_1 m v_2$

On note u se dérive en v : $u \Rightarrow v$.

Dit autrement on a remplacé une variable de u en utilisant une règle de dérivation pour obtenir v .

S'il faut faire n dérivations pour passer de u à v , on note $u \xRightarrow{n} v$.

Définition 2.4 (Langage engendré par G)

C'est l'ensemble des mots que l'on peut obtenir en partant de l'axiome de G et en utilisant les règles de dérivations. C'est à dire $L(G) = \{u, S \xRightarrow{n} u, n \in \mathbb{N}\}$

Définition 2.5 (Mot algébrique)

On appelle mot algébrique tout élément de $(\Sigma \cup V)^$*

Définition 2.6

On appelle langage algébrique d'une grammaire G l'ensemble des mots algébriques obtenable à partir de cette grammaire.

Remarque 2.1. — Usuellement on n'écrit pas $(X, m) \in P$ mais plutôt $X \rightarrow m$.

— De la même manière on ne décrit pas P comme on le ferai pour ensemble, habituellement on liste ses élément en utilisant la notation donnée précédemment.

Par exemple :

$$\begin{aligned} \cdot S &\rightarrow aS \\ S &\rightarrow \epsilon \end{aligned}$$

Exemple 2.2. Le langage engendré par G_1 est $L(G_1) = \{a^n, n \in \mathbb{N}\}$.

Définition 2.7 (Arbre de dérivation)

Un arbre de dérivation d'un mot v est un arbre A dont la racine est S et dont les feuilles luent de gauche à droite forment le mot v . Les nœuds sont des mots algébriques et les feuilles des mots. L'ensemble des fils d'un nœud forment, une fois concaténés de gauche à droite, un mot en lequel il se dérive. Il peut y avoir plusieurs arbre de dérivation pour un même mot.

Exemple 2.3. Voici l'arbre de dérivation du mot aaa .

Pour générer notre jeu de données il va donc falloir trouvé une grammaire G tel que le langage engendré par G soit l'ensemble des problèmes de probabilité possible. Afin de simplifier cette tâche on peut se contenter d'un sous-ensemble représentatif.

2.2 Implémentation dans un algorithme

2.2.1 Plan de conception

Pour concevoir l'algorithme il faut dans un premier temps trouver la bonne grammaire, ensuite il faut trouver comment l'implémenter en python. Il faut aussi être capable de trouver la réponse au problème générer.

Il est très ambitieux de vouloir créer du premier coup un algorithme qui génère des *bons* problèmes. Il est nettement plus simple de créer un algorithme qui génère un problème simple puis de complexifier au fur et à mesure l'algorithme.

Dans cette optique là, on va se restreindre à des problèmes avec n atomes, n étant un paramètre de l'algorithme.

De plus toujours dans une volonté de simplification, on veut dans un premier temps créer un algorithme qui est capable de générer des problèmes probabilistes. On entend par là que l'on s'autorise à ce que l'algorithme génère des problèmes non valide. Si l'on est capable de déterminer si un problème est valide il n'y ici aucun problème.

Enfin on va se limité à des problèmes probabilistes énoncés sous la forme suivante.

Photo énoncé

Avec toutes ces simplifications il est maintenant beaucoup plus aisé de trouver une grammaire qui génère notre problème.

2.2.2 Grammaire utilisée

On va couper la génération en deux. D'un côté on va générer les prémices et de l'autre la question.

$$\cdot S \rightarrow \text{premise.} \backslash n \text{ question.}$$

Comme on veut avoir potentiellement plus qu'un prémice, le prémice peut se dériver en deux choses différentes : une phrase ou une phrase + lui-même. Ainsi par récursivité le nombre de prémice possible est non-borné.

$$\cdot S \rightarrow \text{premise.} \backslash n \text{ question.}$$

$$\cdot \text{premise} \rightarrow \text{Une phrase.}$$

$$\text{premise} \rightarrow \text{Une phrase.} \backslash n \text{premise}$$

Quitte à faire reformuler plus tard notre problème par un LLM (par exemple ChatGPT) on peut donner la même forme aux phrases. Par exemple *Il y a une probabilité de proba_i d'être event*, proba_i désigne un nombre entre 0 et 1 dépendant de i et *event* désigne une formule logique dont les littéraux sont des atomes.

On se donne donc une grammaire de la forme.

- $S \rightarrow \text{premise.} \backslash n \text{ question.}$
- $\text{premise} \rightarrow \Pi \text{ y a une probabilité de } \text{proba}_i \text{ d'être } \text{event.}$
 $\text{premise} \rightarrow \Pi \text{ y a une probabilité de } \text{proba}_i \text{ d'être } \text{event.} \backslash n \text{ premise}$
- $\text{event} \rightarrow \text{expr}$
 $\text{event} \rightarrow \text{expr sachant expr}$
- $\text{expr} \rightarrow \text{and}$
 $\text{expr} \rightarrow \text{or}$
 $\text{expr} \rightarrow \text{color}$
- $\text{and} \rightarrow \text{expr}$
- $\text{or} \rightarrow \text{expr}$
- $\text{color} \rightarrow \text{blue}$
 $\text{color} \rightarrow \text{red}$
 $\text{color} \rightarrow \text{green}$
 $\text{color} \rightarrow \text{orange}$

On a pris comme atomes le fait d'être d'une couleur. Comme on peut toujours remplacer après-coup les couleurs par d'autres événements on ne perd pas en généralité.

Pour les même raisons que précédemment, on donne une forme caractéristique à la question : 'Quelle est la probabilité d'être *expr*?'.

- $S \rightarrow \text{premise.} \backslash n \text{ question.}$
- $\text{premise} \rightarrow \Pi \text{ y a une probabilité de } \text{proba}_i \text{ d'être } \text{event.}$
 $\text{premise} \rightarrow \Pi \text{ y a une probabilité de } \text{proba}_i \text{ d'être } \text{event.} \backslash n \text{ premise}$
- $\text{event} \rightarrow \text{expr}$
 $\text{event} \rightarrow \text{expr sachant qu'on est expr}$
- $\text{expr} \rightarrow \text{and}$
 $\text{expr} \rightarrow \text{or}$
 $\text{expr} \rightarrow \text{color}$
- $\text{and} \rightarrow \text{expr}$
- $\text{or} \rightarrow \text{expr}$
- $\text{color} \rightarrow \text{blue}$
 $\text{color} \rightarrow \text{red}$
 $\text{color} \rightarrow \text{green}$
 $\text{color} \rightarrow \text{orange}$
- $\text{question} \rightarrow \text{Quelle est la probabilité d'être } \text{expr} ?$
 $\text{question} \rightarrow \text{Quelle est la probabilité d'être } \text{expr sachant qu'on est expr} ?$

Avec cette grammaire on obtient déjà quelque chose de bien, on peut cependant l'améliorer pour augmenter la probabilité d'avoir des problèmes intéressants. Par exemple on peut limiter le nombre de *ou* et de *et* au sein d'une même phrase. On veut aussi éviter les *expr* de la forme *green or green*. Pour cela il faut traquer les dérivations que l'on réalise. Pour ce faire on peut rajouter une variable *data* à chaque mot générer par notre grammaire. Si l'on adopte le point de vue de l'arbre de dérivation, cela signifie que chaque nœud possède une variable supplémentaire (en plus de son étiquette).

Il ne reste plus qu'à remplacer les *proba_i* par des valeurs cohérentes pour obtenir un problème probabiliste juste. Une manière naïve de faire cela est de prendre des valeurs aux hasards. On peut alors espérer que parmi *m* problèmes générés il y en ai au moins 1 de bien posé, avec *m* potentiellement très grand. Ce n'est pas du tout un problème si *m* > 1 000 000 puisque que l'on capable de paralléliser la génération. Par exemple si *m* = 1 000 000, que l'on a 1 000 machines à disposition et que générer un problème prend 0.001 secondes en moyenne, il faudra 10 secondes pour générer un problème juste.

2.2.3 Utilisation de problog

La grammaire de la sous-partie précédente génère des problèmes probabilistes, cependant elle ne donne pas la solution.

Pour l'obtenir on peut utiliser des solveurs. Dans le cadre du stage, j'ai utilisé Problog. C'est un solveur de problèmes probabilistes basé sur le langage Prolog.

Pour obtenir la solution à un problème, il faut le coder dans le langage du solveur puis faire tourner le code. On souhaite donc générer automatiquement le problème dans le langage Problog.

On ne peut cependant pas procéder comme précédemment et créer une nouvelle grammaire. En effet en faisant cela on générerait un problème probabiliste en langage Problog mais celui-ci n'aurait a priori aucun lien avec celui généré par l'appel à la grammaire précédente.

Pour résoudre ce problème on peut créer une grammaire *double*. Au lieu de créer une grammaire sur $\Sigma = \{a, b, \dots, z\}$, on le fait sur $\Sigma \times \Sigma$. Le premier élément du couple correspond à notre problème énoncé en langage naturel et le deuxième élément est la version Problog du même problème.

Exemple 2.1

Par exemple si l'on considère le problème suivant :

Un grand magasin compte 500 employés. Il y a 350 femmes, dont 200 ont moins de 25 ans. Il y a 75 hommes de moins de 25 ans. Si un employé est sélectionné pour une promotion, trouvez la probabilité que cet employé ait plus de 24 ans et soit une femme.

On peut le coder en Problog de la manière suivante :

```

1 :- use_module(library(nlp4plp)).
2
3 group(1-7).
4 size(1-7, 500).
5
6 given(exactly(200, 1-7, and(25, female))).
7 given(exactly(75, 1-7, and(25, male))).
8 given(exactly(350, 1-7, female)).
9
10 take(1-7, 4-3, 1).
11
12 probability(and(none(4-3, 25), all(4-3, female))).
13
14 property(gender, [male, female]).
15 property(age, [25]).

```

Une fois exécuté, on obtient 0.3 comme réponse, ce qui est le bon résultat.

En utilisant Problog et la technique de la grammaire *double* on obtient :

- $S \rightarrow \text{setup.} \backslash \text{n take.} \backslash \text{n question}$
- $\text{setup} \rightarrow \begin{cases} \text{group}(\text{groupName}). \backslash \text{n size}(\text{groupName}, \text{groupSize}). \backslash \text{n premise} \\ \text{Soit } \text{groupName} \text{ un ensemble de } \text{groupSize} \text{ } \text{memberName}. \backslash \text{n premise} \end{cases}$
- $\text{premise} \rightarrow \begin{cases} \text{given}(\text{exactly}(\text{rel}(\text{proba}_i, \text{groupName}), \text{groupName}, \text{expr})). \\ \text{Il y a une probabilité de } \text{proba}_i \text{ d'être } \text{expr}. \end{cases}$
- $\text{premise} \rightarrow \begin{cases} \text{given}(\text{exactly}(\text{rel}(\text{proba}_i, \text{groupName}), \text{groupName}, \text{expr})). \backslash \text{n premise} \\ \text{Il y a une probabilité de } \text{proba}_i \text{ d'être } \text{expr}. \backslash \text{n premise} \end{cases}$
- $\text{premise} \rightarrow \begin{cases} \text{given}(\text{exactly}(\text{rel}(\text{proba}_i, \text{groupName}), \text{expr}, \text{expr})). \\ \text{Il y a une probabilité de } \text{proba}_i \text{ d'être } \text{expr} \text{ sachant qu'on est } \text{expr}. \end{cases}$
- $\text{premise} \rightarrow \begin{cases} \text{given}(\text{exactly}(\text{rel}(\text{proba}_i, \text{groupName}), \text{expr}, \text{expr})). \backslash \text{n premise} \\ \text{Il y a une probabilité de } \text{proba}_i \text{ d'être } \text{expr} \text{ sachant qu'on } \text{expr}. \backslash \text{n premise} \end{cases}$
- $\text{expr} \rightarrow \text{and}$
- $\text{expr} \rightarrow \text{or}$
- $\text{expr} \rightarrow \text{color}$
- $\text{and} \rightarrow \begin{cases} \text{and}(\text{expr}, \text{expr}) \\ \text{expr et expr} \end{cases}$
- $\text{or} \rightarrow \begin{cases} \text{or}(\text{expr}, \text{expr}) \\ \text{expr ou expr} \end{cases}$
- $\text{color} \rightarrow \text{blue}$
- $\text{color} \rightarrow \text{red}$
- $\text{color} \rightarrow \text{green}$
- $\text{color} \rightarrow \text{orange}$

- $question \rightarrow \begin{cases} question(exactly(1, groupNameBis, expr)) \\ \text{Quelle est la probabilité qu'il soit } expr? \end{cases}$
- $take \rightarrow \begin{cases} take(groupName, groupNameBis, 1). \\ \text{Choisissons une personne dans le groupe } groupName \end{cases}$

2.3 Résultats obtenus

Le code du programme est disponible en annexe. En lançant le programme on obtient le résultat suivant :

```

1
2
3 Let $groupName a set of $groupSize $memberName.
4
5 $propPercent8 percents of $memberName s who are orange are white
6
7 Let's pick $countBis $memberName from $groupName.
8 What is the probability that more than $count $memberName are orange ?
9
10 :- use_module(library(nlp4plp)).
11
12 group(groupName).
13 size(groupName, 25).
14 given(exactly(rel(0.5432943938875069, groupName, orange), groupName, white)).
15
16 take(groupName, groupNameBis, 5).
17 probability(more_than(2, groupNameBis, orange)).
18 property(color, [blue, red, white, green, black, orange]).
19 property(emotion, [happy, sad, angry]).
20 property(clothes, [wearing_shoes, wearing_glasses, wearing_a_shirt]).

```

L'algorithme peut se résumer par le schéma suivant :

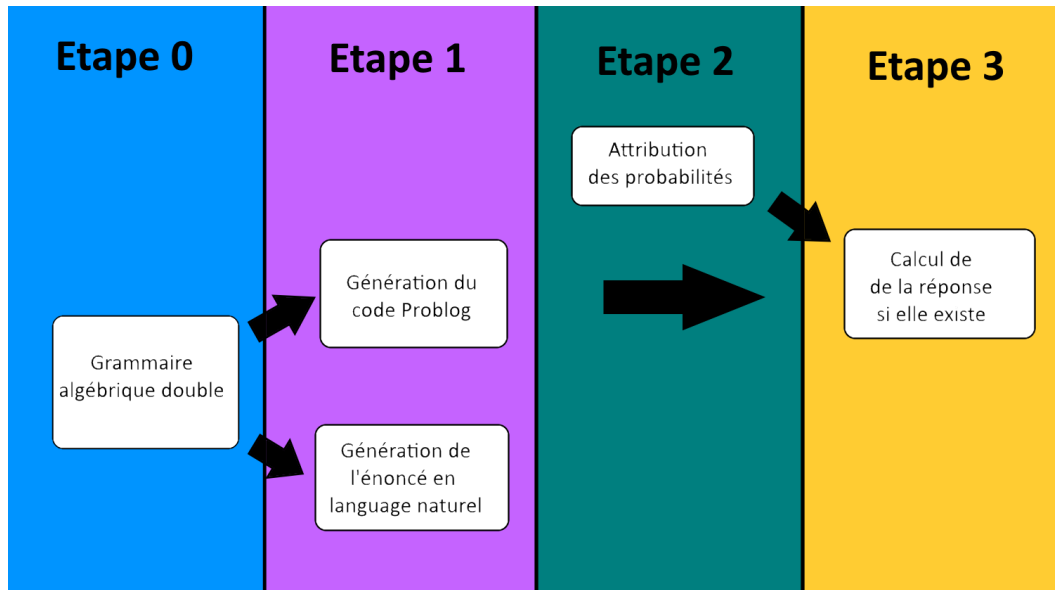


FIGURE 1 – Schéma récapitulatif de l'algorithme

2.4 Problèmes

Cet algorithme est capable de générer des problèmes probabilistes justes mais très souvent les problèmes créés comportent des erreurs ou des incohérences. Comme expliquer précédemment ce n'est pas problématique si l'on est capable de différencier les problèmes valides des non-valides.

Les problèmes générés peuvent être insolubles. Par exemple :

*La probabilité que quelqu'un porte du vert est 0.4
 Quelle est la probabilité que quelqu'un porte du bleu ?*

Les prémices peuvent aussi être incohérentes. Par exemple :

*La probabilité que quelqu'un porte du vert est 0.4.
 La probabilité que quelqu'un porte du noir est 0.3.
 La probabilité que quelqu'un porte du vert et du noir est 0.5.
 Quelle est la probabilité que quelqu'un porte du bleu ?*

Il n'est pas possible d'avoir $P(\text{Vert} \cap \text{Noir}) > P(\text{Vert})$

Enfin le plus gros problème se situe au niveau de Problog. Lorsqu'il reçoit plus d'information que nécessaire pour résoudre un problème, il donne parfois la mauvaise réponse.

*La probabilité que quelqu'un porte du vert est 0.4.
 La probabilité que quelqu'un porte du vert est 0.4.
 Quelle est la probabilité que quelqu'un porte du vert ?*

Problog ne donne pas 0.4 comme réponse.

De plus Problog donne parfois des réponses à des problèmes mal posés. On ne peut donc pas utiliser Problog pour séparer les problèmes justes des autres problèmes.

Il n'est donc pas possible de générer un jeu de donné à l'aide de cet algorithme.

3 Deuxième algorithme

On a vu dans la partie précédente que l'algorithme ne fonctionnait pas à cause de Problog. Pour contourner le problème, on peut donc essayer de créer un solveur personnalisé en s'inspirant de Problog. Cela offre plusieurs avantages : un calcul des résultats plus simple car directement codé en python, des résultats plus fiables. Enfin en utilisant ce solveur personnalisé à chaque étape de la génération du problème, il est possible d'écrire un algorithme que ne produit que des problèmes bien posés.

3.1 Graphe bayésien

Problog ainsi que la plupart des solveurs de problème probabilistes reposent sur une notion : les graphes bayésiens. Ils génèrent le graphe bayésien lié au problème et l'utilise pour calculer les probabilités inconnues.

3.1.1 Définition

Définition 3.1 (Graphe orienté)

Un graphe orienté G est un couple $G = (V, E)$ où V est un ensemble appelé sommets ou vertices et E est un sous-ensemble de V^2 . Les éléments de E sont appelés des arcs.

Exemple 3.1. On représente en général les graphes de la manière suivante.

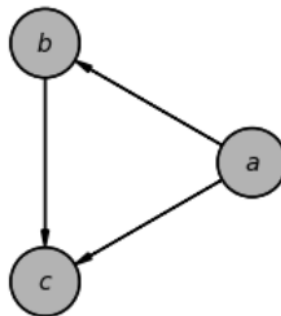


FIGURE 2 – Exemple de graphe orienté

Définition 3.2. Soit G un graphe.

On appelle parents du nœud n l'ensemble des nœuds p de G tel qu'il existe un arc de p vers n .

On appelle enfants / fils du nœud n l'ensemble des nœuds dont n est un parent.

Définition 3.3. On appelle racine du graphe tout nœud n'ayant pas de parent.

Définition 3.4 (Graphe cyclique)

Un graphe G est dit cyclique s'il existe des sommets $v_1, \dots, v_n$ tels que $\forall i \in \{1, \dots, n-1\}, (v_i, v_{i+1}) \in E$ et $(v_n, v_1) \in E$. Un graphe non cyclique est dit acyclique.

Exemple 3.1

Le graphe suivant est cyclique.

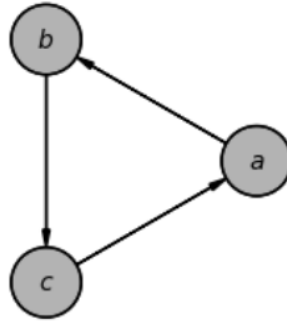


FIGURE 3 – Exemple de graphe cyclique

3.1.2 Hypothèses

Définition 3.5 (Graphe causal)

Un graphe causal est un graphe orienté acyclique dont les nœuds sont des variables aléatoire et qui vérifie les trois hypothèses suivantes :

- causal markov
- faithfulness
- causal sufficiency

Il est utilisé pour représenté les liens causaux entre les événements d'un problème probabiliste.

Remarque 3.1. Étant donné que dans la suite du rapport les variables utilisées seront exclusivement des indicatrices d'événements, on assimilera les nœuds aux événement.

Hypothèse 1 : causal markov

Un nœud X est indépendant de tous ses non-descendants sachant ses parents.

Exemple 3.2

Sur le graphe suivant :

- $C|A$ et $B|A$ sont indépendants
- $D|B, C$ est indépendants de A

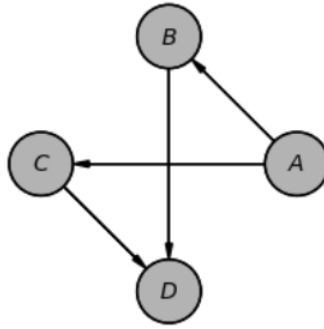


FIGURE 4 – Exemple causal markov

Hypothèse 2 : faithfulness

Les arcs du graphe et son hypothèse causal markov représente fidèlement l'ensemble des liens de causalité direct entre les événements. Un arc va du nœud A vers le nœud B si et seulement si A influence directement B .

Exemple 3.3

Considérons un exemple et deux graphes, un qui marche et l'autre non

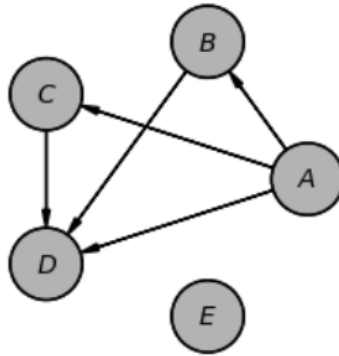


FIGURE 5 – Exemple faithfulness

Hypothèse 3 : causal sufficiency

L'ensemble des causes sont observées et par conséquent représentées dans le graphe.

Définition 3.6 (Graphe bayésien)

Un graphe bayésien est un graphe causal dont chaque nœud comporte une table. Cette table décrit la probabilité de l'événement attribué au nœud sachant ses causes directes i.e. les parents du nœud.

Exemple 3.4

Un graphe bayésien avec 3 événements :

3.1.3 Utilisation des graphes bayésiens

Proposition 3.1

Le graphe bayésien décrit entièrement le problème probabiliste et permet de calculer la probabilité de tous les événements.

Méthode de calcul de la probabilité d'un événement à l'aide d'un graphe bayésien

Pour calculer la probabilité d'un événements on utilise la loi de Bayes :

Définition 3.7. Soit A et B deux événements : $P(A \wedge B) = P(A|B)P(B)$

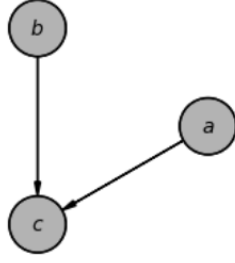


FIGURE 6 – Graphe bayésien

	$\neg A$		A	
	$\neg B$	B	$\neg B$	B
$P(C)$	0.1	0.8	0.4	0.2
$P(\neg C)$	0.9	0.2	0.6	0.8

$P(A)$	0.4	$P(B)$	0.3
$P(\neg A)$	0.6	$P(\neg B)$	0.7

FIGURE 7 – Tables du graphe

1er cas : La probabilité des causes est connue

On calcule à l'aide de la loi de Bayes la probabilité de notre événement sachant ses causes dans chacune des configuration possible du système

Dans l'exemple ci-dessus, on a :

$$\begin{aligned}
 P(C) &= P(C|A \wedge B)P(A \wedge B) + P(C|A \wedge \neg B)P(A \wedge \neg B) \\
 &\quad + P(C|\neg A \wedge B)P(\neg A \wedge B) + P(C|\neg A \wedge \neg B)P(\neg A \wedge \neg B) \\
 &= P(C|A \wedge B)P(A)P(B) + P(C|A \wedge \neg B)P(A)P(\neg B) \\
 &\quad + P(C|\neg A \wedge B)P(\neg A)P(B) + P(C|\neg A \wedge \neg B)P(\neg A)P(\neg B)
 \end{aligned}$$

La deuxième égalité découle de l'hypothèse causal markov qui assure l'indépendance de deux nœuds n'ayant pas d'ancêtre commun.

Plus généralement si l'on considère un événement B de causes directes $A_1, \dots, A_n$:

$$P(B) = \sum_{I \in \mathcal{P}(\{1, \dots, n\})} P\left(B \mid \bigwedge_{i \in I} A_i \wedge \bigwedge_{i \notin I} \neg A_i\right) \underbrace{\prod_{i \in I} P(A_i) \prod_{i \notin I} P(\neg A_i)}_{\text{probabilité d'avoir une configuration de cette forme}}$$

2ème cas : La probabilité des causes n'est pas connu

A l'aide du cas précédent, montrons par récurrence forte sur la distance du plus loin ancêtre p des nœuds que l'on peut calculer la probabilité de chaque événement présent dans le graphe.

- $p = 0$ Le nœud est alors une racine, sa probabilité est donc explicitement donnée dans la table de probabilité.
- Soit $p \in \mathbb{N}$. Supposons que l'hypothèse de récurrence soit vérifiée pour tout $k \in \{0, \dots, p\}$. Soit n un nœud dont le plus loin ancêtre est à une distance $p + 1$ dans le graphe. Par ce qui précède :

$$P(n) = \sum_{I \in \mathcal{P}(\{1, \dots, r\})} P\left(n \mid \bigwedge_{i \in I} A_i \wedge \bigwedge_{i \notin I} \neg A_i\right) \prod_{i \in I} P(A_i) \prod_{i \notin I} P(\neg A_i)$$

Comme les ancêtres des A_i sont les ancêtres de n et que si C est un ancêtre de A_j à une distance k , il est un ancêtre de n avec une distance $k + 1$, les ancêtre des A_i sont à une distance inférieur à p .

D'après l'hypothèse de récurrence, on peut calculer à l'aide du graphe les $P(A_i)$. Par conséquent, $P(n)$ est calculable à partir du graphe.

- Par récurrence forte, on a le résultat pour tout élément du graphe.

3ème cas : Calcul de la probabilité d'un événement quelconque

Supposons que notre graphe à m événements observés $A_1, \dots, A_m$. On peut créer un nouvel événement à partir de ces événements de base et des opérations booléennes. Par exemple $A \cap B$ ou $A \cup B$. De manière plus générale, on peut se donner une formule logique ϕ ayant les m événement de base

pour variable, $B = \phi(A_1, \dots, A_m)$ est alors un nouvel événement.

Pour calculer $P(B)$, on peut rajouter un noeud B au graphe qui admet pour parent les autres noeud graphe. Pour sa table de probabilité on pose :

	$\neg\phi(A_1, \dots, A_m)$	$\phi(A_1, \dots, A_m)$
$P(B)$	0.0	1.0
$P(\neg B)$	1.0	0.0

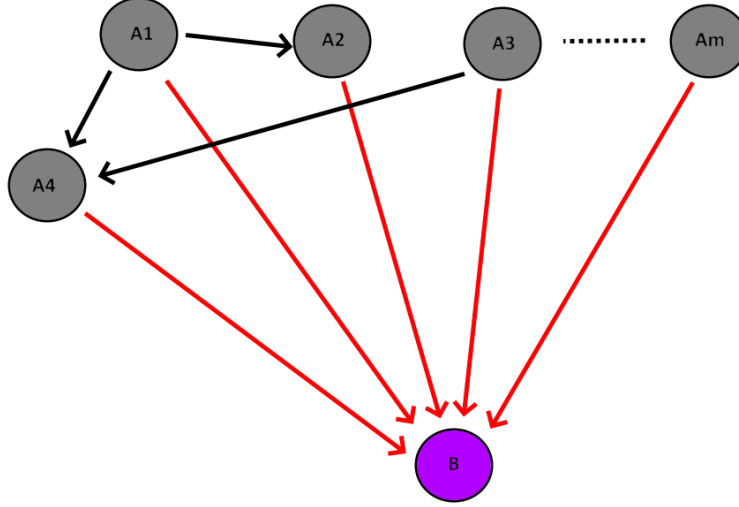


FIGURE 8 – Ajout du noeud B

Ainsi $P(B) = P(\phi(A_1, \dots, A_m))$ = probabilité que le système rende ϕ vrai.
Or par ce qui précède, $P(B)$ est calculable en tant que noeud du graphe.
Donc tout les événements sont calculables à l'aide du graphe.

Utilités des graphes bayésiens pour générer des probabilités

Les graphes bayésiens sont non seulement très utiles pour calculer des probabilités mais ils peuvent aussi être utilisés pour en générer.

Comme vu en partie 2, lorsque l'on génère des probabilités pour des événements non indépendants, il y a des risques d'incohérence. Les graphes bayésiens permettent de contourner ce problème. En effet les événements conditionnés par leur causes directes sont indépendants de tous les autres événements du graphes (hypothèse causal markov).

EXPLIQUER PK ON PEUT PAS SE TROMPER

3.1.4 Limites

Les graphes bayésiens sont très pratiques puisqu'il permette de représenter facilement un ensemble d'événement, leur probabilité et les liens causaux qui les relient. La probabilité de chaque événement peut être calculer à l'aide du graphe.

Cependant il y a des inconvénients. Premièrement la structure prend beaucoup de place en mémoire. La taille d'une table d'un noeud est en 2^p où p est le nombre de parent du noeud. Autre inconvénient, la structure ne fait pas apparaître explicitement la probabilité des prémices. Enfin, le graphe doit être complètement remplie pour être fonctionnelle. Dit autrement, le graphe peut avoir besoin d'informations superflu pour calculer la probabilité d'un événement.

Exemple 3.5

Supposons que l'on ai trois événements : A, B, C .

On connaît les liens causaux entre A, B et C , ils se représentent par le graphe suivant :

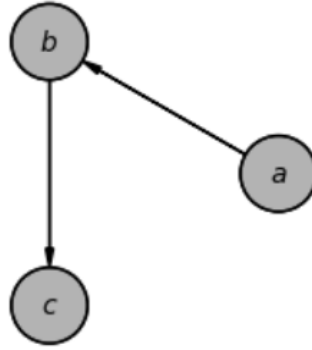


FIGURE 9 – Graphe causal du problème

Supposons que l'on connaisse $P(C|A)$ et $P(C|\bar{A})$, on peut alors calculer $P(C)$. Toutefois il n'est pas possible de remplir les tables du graphe bayésien lié au problème, et par extension de calculer $P(C)$.

3.2 Implémentation dans un algorithme

Etape 1 : Générer le graphe

Pour générer le graphe on va utiliser la bibliothèque *pgmpy*.

Elle permet de générer des graphes bayésiens aléatoires.

Tout comme dans le premier algorithme, on va fixer le nombre d'événement de base. Pour le nom des événement on peut se contenter de $A_1, \dots, A_m$ mais j'ai trouvé que c'était plus agréable de travailler avec des noms concrets, cela donnera du sens aux phrases que l'on générera. Il n'y aura pas besoin de passer par une phase de remplacement des noms des événements pour que le problème ai un sens. J'ai donc choisi comme nom des couleurs, des émotions et le fait de porter certains vêtements.

couleurs	émotions	attributs vestimentaire
blue	happy	wearing_glasses
red	sad	wearing_shoes
white	angry	wearing_a_shirt

Pour générer le graphe, j'ai codé le code suivant :

```
1 from pgmpy.models import BayesianNetwork
2
3 l1 = ['blue', 'red', 'white']
4 l2 = ['happy', 'sad', 'angry']
5 l3 = ['wearing_glasses', 'wearing_shoes', 'wearing_a_shirt']
6 l = l1 + l2 + l3
7 length = len(l)
8
9 G = BayesianNetwork.get_random(n_nodes=length, n_states=2) #Genere un graphe bayesien
   avec length noeud dont chaque noeud a 2 etats differents. Les tables de probabilite
   sont vides
10 G.get_random_cpds(n_states=2, inplace=True) #Genere les tables de probabilite
11 G.to_daft(node_pos="circular").render() #Dessine le graphe
```

On obtient :

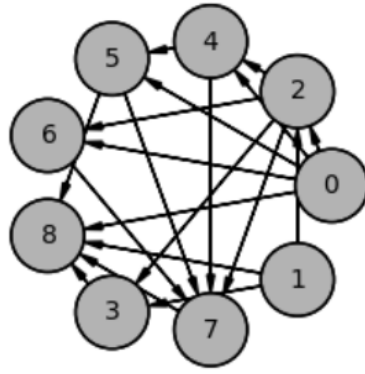


FIGURE 10 – Graphe obtenu

EXPLIQUER PK LES PROBA SONT BONNES

Grâce au graphe on a un ensemble d'événement probabiliste. Contrairement à l'algorithme 1, on peut choisir aléatoirement les valeurs dans les tables de probabilité. Il faut juste que la somme dans chaque colonne soit 1.

Etape 2 : Choisir un noeud

On va chercher à générer des question de la forme : *Quelle la probabilité de A sachant B ?*.

Pour cela on va choisir deux nœuds dans notre graphe.

On choisit aléatoirement le premier nœud (le A).

Pour le deuxième nœud, on a deux choix :

1. choisir le nœud parmi les ancêtre ou les descendant de A afin de garantir que $P(A|B) \neq P(A)$.
De cette manière la réponse naïve à la question n'est pas la bonne réponse. Toutefois lors de l'entraînement du réseau de neurone, il est possible qu'il 'apprenne' que l'on conditionne toujours par un événement qui n'est pas indépendant. Dit autrement cette méthode fait perdre en généralité.
2. choisir aléatoirement le nœud B dans le graphe.

Le deux méthodes sont biens, l'idéal serait d'utiliser les deux. Dans un premier temps on utilise la méthode 2 qui est général. Après cela, il est possible d'utiliser la 1 pour garantir un certain nombre de question *plus difficile*. De cette manière il est possible de générer tout type de question en garantissant la présence de certaines questions.

Cas particulier $A = B$

Ce cas particulier se produit relativement souvent ($(\frac{1}{9})^2 > 1\%$ dans notre cas). Il peut être intéressant d'avoir certaines questions de la forme $P(A|A)$, pour entrainer notre réseau à réaliser qu'il s'agit juste de $P(A)$. En pratique, personne ne posera à notre réseau ce genre de question. Il n'est donc pas nécessaire d'en avoir beaucoup dans notre data-set.

Une des solutions consiste à remplacer 95% de ces questions par de simple question de la forme $P(A)$ (actuellement pas générées) qui elles risquent d'être posées régulièrement.

Dans le cas 1 :

```

1 i = random.randint(0, length - 1) #noeud 'A'
2
3 H = G.get_ancestral_graph(i) #on recupere les ancetres
4
5 ns = list(H.nodes())
6 n = random.choice(ns) #on choisit aux hasard un element

```

Dans le cas 2 :

```

1 i = random.randint(0, length - 1) #noeud 'A'
2 n = random.randint(0, length - 1) #noeud 'B'

```

Etape 3 : Générer les prémices

Il faut générer les phrases qui vont décrire notre problème. Comme dans le reste du rapport on va d'abord utiliser une méthode naïve que l'on perfectionnera.

On va parcourir les tables de probabilité de chaque nœud et énoncer le contenu. On va utiliser des phrases de la forme : *La probabilité de A sachant B est x.*

Comme les tables contiennent $P(A|B)$ et $P(\neg A|B)$ il n'est pas nécessaire d'énoncer la table dans son entièreté pour être exhaustif. On va donc choisir aléatoirement à chaque fois si on prend l'événement ou son contraire.

Cette partie de l'algorithme est longue et va beaucoup changer, le code final est donné plus tard dans le rapport.

Etape 4 : Poser la question et calculer la réponse

On applique ce qui est énoncé à l'étape 2.

```
1 from pgmpy.inference import VariableElimination
2
3 question = ''
4 answer = 0
5
6 inf = VariableElimination(G)
7
8 if i != n or random.random() > 90:
9     phi_query = inf.query([i], [n])
10    answer = phi_query.values((1,))
11    question = 'What is the probability that Damien is 1[i] given 1[n] ?'
12 else :
13     phi_query = inf.query([i], [])
14     answer = phi_query.values((1,))
15     question = 'What is the probability that Damien is 1[i] ?'
16
17 print(gen_premices(G, i, n))
18 print(question)
19 print(f"Answer: {answer}")
```

3.3 Résultats

On lançant l'algorithme on obtient le résultat suivant :

```
1 If someone is not wearing_color, then the probability that they are not blue is
   0.44229670469666166
2 If someone is wearing_color, then the probability that they are not blue is
   0.44229670469666166
3 If someone is not blue and not wearing_color, then the probability that they are
   white is 0.40461687860698375
4 If someone is blue and not wearing_color, then the probability that they are white
   is 0.40461687860698375
5 If someone is wearing_color and not blue, then the probability that they are not
   white is 0.5953831213930163
6 If someone is blue and wearing_color, then the probability that they are not white
   is 0.5953831213930163
7 If someone is not wearing_color, then the probability that they are red is
   0.5577032953033384
8 If someone is wearing_color, then the probability that they are red is
   0.5577032953033384
9 There is a probability of 0.36186177420161986 that Neimad is wearing_color today.
10 What is the probability that Damien is white given he is wearing_color ?
11 Answer: 0.40461687860698375
```

Si l'on donne le problème à Chat-gpt on obtient :

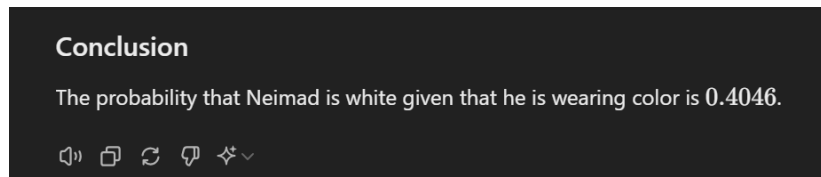


FIGURE 11 – Réponse de Chat-gpt

4 Améliorations

L'algorithme génère bien des problèmes valides avec leur réponse. Toutefois des LLM comme Chat-gpt sont capable de résoudre les problèmes créés par mon algorithme avec un faible pourcentage d'erreur. Il faut donc complexifier les problèmes générés.

Il existe plusieurs manières de réaliser cela.

1. Augmenter la taille du problème en rajoutant plus d'événement. Cette solution est peu efficace car Chat-gpt *connait* la loi de Bayès, s'il est capable de l'appliquer 10 fois de suite, il y a de forte chance qu'il réussisse à le faire 20 fois de suite. Toutefois on devrait quand même voir augmenter le pourcentage de réponse loin du résultat exacte car les LLM sont très mauvais pour faire des additions et des multiplications avec des nombres ayant beaucoup de décimal.
2. Faire en sorte que l'utilisation seule de la loi de Bayès ne permette pas de résoudre les problèmes. Dit autrement, augmenter les compétences nécessaires à la résolution des problèmes.

Exemple 4.1

Lorsque l'on demande à Chat-gpt de calculer :

$$1,08754533667543 * 2,6439998765325788$$

On obtient 2,877,332,267,499,731,325,263,430,015.1405, ce qui n'est pas du tout la bonne réponse pour deux raisons :

- Il n'a pas bien compris ce que la virgule représente en français.
- Les chiffres sont mauvais, une approximation du résultat est : 2,87546973589

Pour réaliser l'idée 2 on peut :

1. Rajouter la notion de groupe. Il ne s'agirait plus de trouver la probabilité qu'une personne ait tel caractéristique mais plutôt qu'au sein d'un sous groupe donné, il y ait au moins / au plus / exactement k personnes ayant cette caractéristique.
2. Appliquer des lois de probabilité pour substituer certaines prémices.
Par exemple, remplacer $P(A)$ par $P(A \cup B)$, $P(B)$ et $P(A \cap B)$ à l'aide de l'égalité : $P(A \cup B) = P(A \cap B) + P(A) + P(B)$
3. Faire du do-calculus. La notion est expliciter ci-dessous.

4.1 Complexification des prémices

Comme expliqué précédemment, pour complexifier les prémices, on va fonctionner par substitution en utilisant des égalités connues.

Après la génération des prémices (Etape 3), on se retrouve avec une liste de phrase. Ce n'est pas simple d'extraire la logique de ces phrases et de faire de la substitution avec.

Pour remédier à ce problème, on peut inventer une nouvelle structure appelée *EquationFinder* qui se chargera de stocker un événement de la forme $A \mid B$ avec A et B des événements quelconques. En général on verra A et B comme des formule logique sous forme SAT dont les variables sont des événements du graphe. En python, on implémentera cela par l'objet *Expression* de la bibliothèque

Sympy.

Il va aussi nous falloir de nouveaux objets pour représenter les équations à notre disposition.

On se donne donc trois nouveaux objets : *EquationHalfTerm*, *EquationTerm*, *Equation*.
EquationHalfTerm décrit une clause de la forme l_1 ou $l_1 \wedge \neg l_2$.

EquationTerm combine deux *EquationHalfTerm* pour former une formule de la forme $l_1|l_2$ ou juste l_1 si seulement l_1 est donné.

Equation est un ensemble d'*EquationTerm*.

Le véritable intérêt de ces objets n'est pas leur capacité à condenser des informations mais leur capacité à les utiliser à travers des fonctions propre à ces objets.

Photo Récapitulative

Etape 3.1 : Utilisation

On va générer une première version de nos prémices sous forme d'une liste d'*EquationFinder*. Ensuite, on choisit au hasard un *EquationFinder*. On sélectionne dans notre liste d'*Equation* une équation qui contient un terme de la forme de notre *EquationFinder*. C'est à dire si notre *EquationFinder* correspond à un événement de la forme $A|B$, on va chercher une *Equation* qui contient un *EquationTerm* de la forme $l_1|l_2$.

Exemple 4.2

Voici deux exemples d'équation :

```
1 eq0 = Equation(ET(E(0)), ET(E(1)), ET(E(0, 1, 1)), ET(E(0, 1, 0)))
2 #P(A \cup B) = P(A) + P(B) - P(A \cap B)
3 eq1 = Equation(ET(E(0), E(1)), ET(E(1), E(0)), ET(E(0)), ET(E(1)))
4 #P(A | B) = P(B | A)*P(A)/P(B)
```

et un exemple d'*EquationFinder*

```
1 from sympy import *
2
3 equation_finder = EquationFinder(parse_expr('red'))
```

La fonction suivante permet, étant donné un prémice sous la forme d'*EquationFinder* de trouver une équation qui permettrait de le substituer.

```
1
2 def fae(equation_finder): #find an equation
3     random.shuffle(equation_list)
4     for equation in equation_list:
5         a = equation.get_term(equation_finder) #donne un terme de l'équation qui a la même
6             forme que l'equationfinder
7         if a is not None:
8             return a, equation
9     return None, None
```

Etape 3.2 Calculer les probabilités des nouveaux prémices

Une fois l'étape de substitution réalisée, il faut calculer les probabilités associées à nos nouveaux prémices. Toutefois ces prémices ne sont pas forcément constitué d'événement du graphe mais plutôt de formule logique fait à partir de ceux-ci. L'objet *VariableElimination* de Sympy n'est pas capable de donner la probabilité de ce genre d'événement. Il est cependant capable de connaître la probabilité d'une valuation de nos événements, i.e par exemple $A_1 = \text{vrai}, A_2 = \text{faux}, \dots, A_n = \text{vrai}$.

Pour trouver la probabilité de notre événements il suffit donc de trouver toutes les valuation le rendant vrai. On peut réduire ce problème au problème $\#$ -SAT.

Définition 4.1 (Problème $\sharp$ -SAT)

Soit ϕ une formule logique sous forme normale conjonctive (CNF), i.e conjonction de disjonction. Le problème $\sharp$ -SAT consiste à trouver l'ensemble des valuation de ϕ qui satisfasse ϕ .

Pour réaliser cela en Python, on va utilisé la bibliothèque *pycosat*. Malheureusement *pycosat* a son propre format, il faut donc convertir les expressions du format *sympy* au format *pycosat*.

```

1 def eat_sympy(expr, atoms): #convertit une expressions sympy
2 #atoms est la liste des variable de l'expression. C'est ce qui va nous permettre de
3 faire le lien entre les variable de la formul en pycosat et celle en sympy
4
5 atoms_dict = {}
6 for i in range(len(atoms)):
7     atoms_dict[atoms[i]] = i
8
9 expr = to_cnf(expr, simplify=True)
10
11 def convert(child): #s'occupe de gérer les négations
12     if isinstance(child, Not):
13         return -(int(atoms_dict[child.args[0]])+1)
14     return int(atoms_dict[child]) + 1
15
16 def parse(e):
17     if isinstance(e, Or):
18         return [convert(sub_child) for sub_child in e.args]
19     return [convert(e)]
20
21 if isinstance(expr, And):
22     return [parse(child) for child in expr.args]
23 return [parse(expr)]
24
25
26 def find_all_valuation(cl_list): #résout le problème  $\sharp$ -SAT
27 solutions = []
28 for solution in pycosat.itersolve(cl_list):
29     solutions.append(solution)
30 return solutions #renvoie l'ensemble des valuations satisfaisant la formule

```

Une fois les valuations trouvée, un simple appel *VariableElimination* nous donne la probabilité.

Etape 4.1

Maintenant que l'on a modifié les prémices, on peut aussi modifier les questions.

Il peut être intéressant poser des questions de probabilité sur un sous-groupe de personne plutôt que sur une personne seule. Par exemple : *Dans un groupe de 5 personne rouge, quelle est la probabilité qu'au moins 3 portent une chemise ?*

Pour ce faire on va générer aléatoirement un sous-groupe. Il faut choisir le nombre de personne et leurs caractéristiques. Pour le nombre un simple *random.randint()* suffit. Pour les caractéristiques, on va créer une formule SAT aléatoire.

```

1 def generate_random_sat_formula(num_vars, num_clauses):
2     """Génère une formule SAT aléatoire"""
3     formula = []
4     for _ in range(num_clauses):
5         clause = []
6         for _ in range(random.randint(1, 3)): # Taille des clause entre 1 et 3
7             var = random.randint(1, num_vars)
8             sign = random.choice([-1, 1]) #si on prend la négation
9             clause.append(sign * var)
10        formula.append(clause)
11    return formula

```

Résultat obtenu :

```

1
2 If someone is not wearing_color, then the probability that they are blue is
   0.5577032953033384

```

```

3 If someone is wearing_color, then the probability that they are not blue is
  0.44229670469666166
4 If someone is wearing_color, then the probability that they are red is
  0.5577032953033384
5 If someone is not wearing_color, then the probability that they are not white is
  0.44229670469666166
6 If someone is wearing_color, then the probability that they are not white is
  0.44229670469666166
7 There is a probability of 0.6381382257983802 that Neimad is not wearing_color today.
8 If someone is red, then the probability that they are not wearing_color is
  0.6381382257983802
9 There is a probability of 0.6381382257983802 that Neimad is not wearing_color today.
10 There is a probability of 0.5577032953033384 that Neimad is red today.
11 We consider 4 people at Neimad's work that are blue or red or white and blue or not
    blue or not white.
12 If someone is not wearing_color, then the probability that they are blue is
    0.5577032953033384
13 If someone is wearing_color, then the probability that they are not blue is
    0.44229670469666166
14 If someone is wearing_color, then the probability that they are red is
    0.5577032953033384
15 If someone is not wearing_color, then the probability that they are not white is
    0.44229670469666166
16 If someone is wearing_color, then the probability that they are not white is
    0.44229670469666166
17 There is a probability of 0.6381382257983802 that Neimad is not wearing_color today.
18 If someone is red, then the probability that they are not wearing_color is
    0.6381382257983802
19 There is a probability of 0.6381382257983802 that Neimad is not wearing_color today.
20 There is a probability of 0.5577032953033384 that Neimad is red today.
21 We consider 4 people at Neimad's work that are blue or red or white and blue or not
    blue or not white.
22 What is the probability that less than 2 people are red ?
23 Answer: 0.4036319223797241

```

Même si de prime abord, le résultat ressemble beaucoup à ce que l'on obtenait avant, maintenant Chat-gpt n'arrive plus à résoudre cela correctement.

The probability that 2 or fewer people are red among the 4 people is approximately 0.5968.

FIGURE 12 –

Cela signifie que si on génère un jeu de données avec cet algorithme, Chat-gpt n'arrivera pas à obtenir un score satisfaisant. Dit autrement, ce jeu de donnée sera capable d'améliorer Chat-gpt.

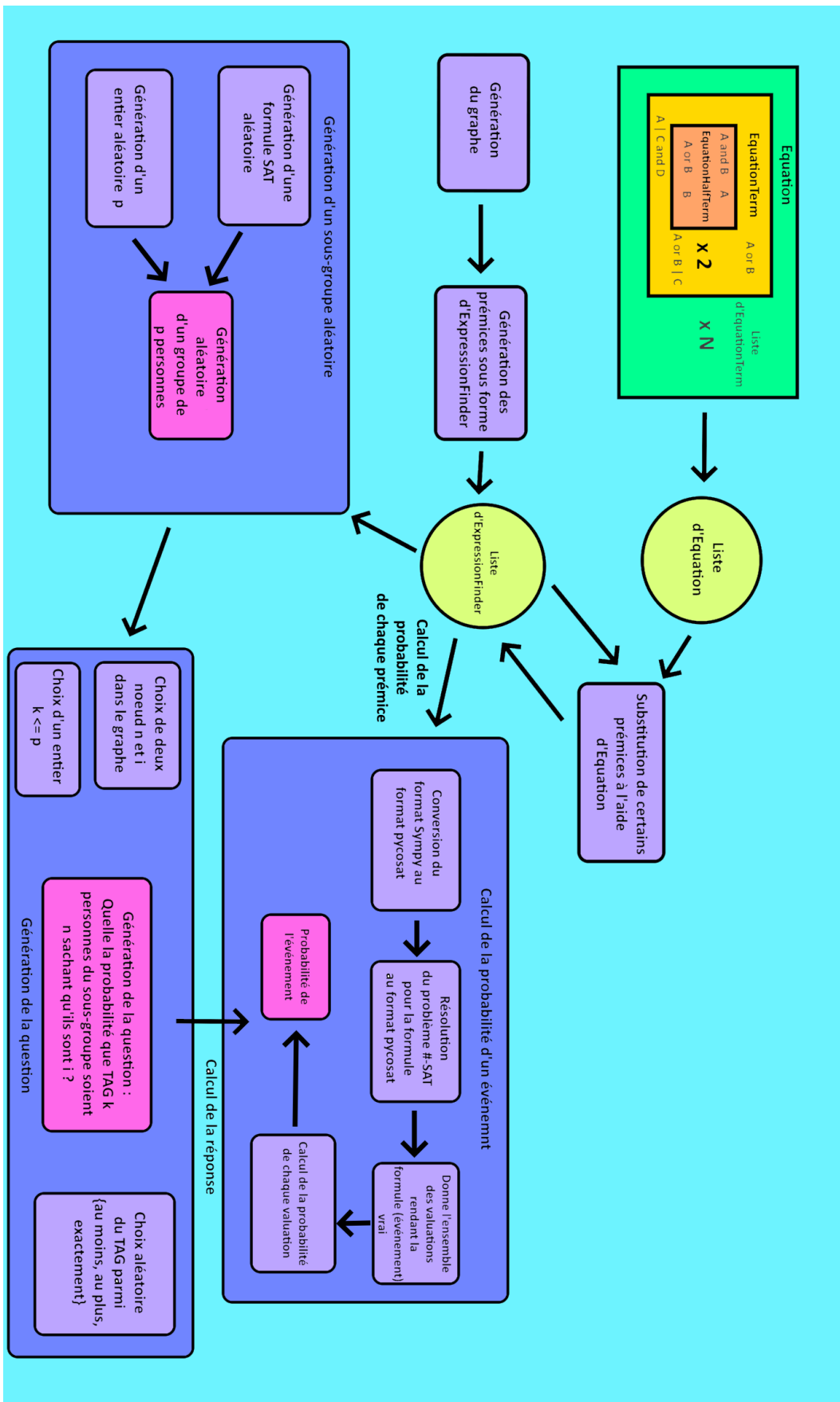


FIGURE 13 – Schéma Récapitulatif

4.2 Ajout du do-calculus

Une autre piste pour améliorer nos problèmes est le *do-calculus*.

Définition 4.2 (Do-calculus)

Le *do-calculus* désigne les problèmes probabilistes posés à l'aide de l'opérateur *do*.

Si A et B sont des événements, $P(\text{do}B)$ désigne la probabilité que A se produit étant donné que B a été réalisé par un opérateur extérieur. Ce n'est pas la même chose que les probabilités conditionnelles.

Exemple 4.3

On considère Jeanne une femme qui a une insuffisance chronique en potassium. Lorsque Jeanne manque de potassium (événement K) elle a mal à la tête (événement T). Pour compenser son manque, Jeanne mange des bananes (événement B) naturellement riches en potassium.

On peut modéliser la situation par le graphe bayésien suivant :

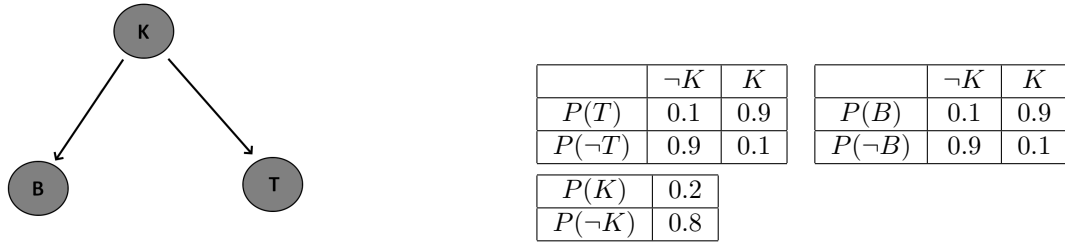


FIGURE 14 – Graphe bayésien du problème

On peut modéliser l'inconfort lié au mal de tête par une variable aléatoire U tel que pour un état $w = K = k, B = b, T = t$,

$$U(w) = b - 20t$$

On confond les événements et leur indicatrice. Plus U est faible plus grand est l'inconfort, on souhaite donc maximiser U .

On peut se demander si manger des bananes réduit effectivement les symptômes, c'est à dire augmenter l'utilité U . Pour ce faire, on va regarder l'espérance de U quand $B = b$ que l'on note $EEU(B = b)$ (Evidential Expected Utility).

$$EEU(B = b) = \sum_w P(w|B = b)U(w)$$

On a :

$$\begin{aligned} P(T|B) &= \frac{P(T \wedge B)}{P(B)} \\ &= \frac{P(B \wedge T|K)P(K) + P(B \wedge T|\neg K)P(\neg K)}{P(B|K)P(K) + P(B|\neg K)P(\neg K)} \\ &\approx 0,65 \end{aligned}$$

et

$$\begin{aligned} P(T|B) &= \frac{P(T \wedge \neg B)}{P(\neg B)} \\ &= \frac{P(\neg B \wedge T|K)P(K) + P(\neg B \wedge T|\neg K)P(\neg K)}{P(\neg|K)P(K) + P(\neg B|\neg K)P(\neg K)} \\ &\approx 0,12 \end{aligned}$$

Ce qui donne :

$$EEU(B = 1) \approx (1 - 20) * 0,65 \approx -12$$

$$EEU(B = 0) \approx -20 * 0,12 \approx -2,4$$

Il est donc plus utile de ne pas prendre de banane que d'en prendre. C'est complètement paradoxale car les bananes sont sensé diminuer les carence en potassium. Cela vient du faire que la prise de banane est très corrélé avec les maux de tête car elles ont une cause commune : la carence en potassium. Cette corrélation est de plus très mal représentée car il peut y avoir une temporalité dans les événements.

Pour obtenir un résultat juste il faut donc dé-corréler les deux.

En réalité ce que l'on cherche c'est $CEU(B = b)$ (Causal Expected Utility), l'espérance de U sachant la valeur B dans le cas où l'on a pris (ou pas) une banane spontanément. Pour ce faire on va faire appel à $P(T \text{ do } B)$ la probabilité que l'on ai T indépendamment des causes de B . Vu autrement, cela revient à trouver $P(T)$ dans le graphe suivant.

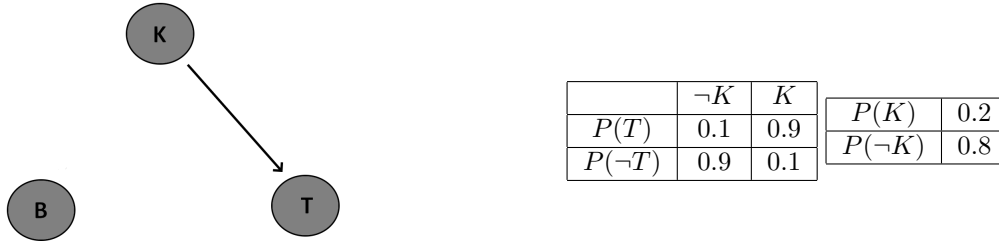


FIGURE 15 – Graphe bayésien du problème

On a

$$CEU(B = b) = \sum_w P(w \text{ do } B = b) U(w)$$

De plus,

$$P(T \text{ do } B = 1) = 0,26$$

$$P(T \text{ do } B = 0) = 0,26$$

Ce qui donne :

$$CEU(B = 1) = -4,2$$

$$CEU(B = 0) = -5,2$$

On trouve bien le résultat attendu, il est plus intéressant de prendre des bananes.

Pour implémenter cela dans notre algorithme, on va choisir un nœud du graphe au hasard et le séparer de ses causes. C'est à dire que l'on va supprimer tous les arcs arrivant sur ce nœud. Désormais ce nœud devient un événement spontané, ou formulé autrement on obtient le graphe *do* ce nœud.

Cela peut être réaliser en utilisant la fonction *do* déjà implémenter dans la bibliothèque pgmpy.

```

1 def gen_do(graph, n, i):
2     nl = list(graph.nodes())
3     node = random.choice(nl)
4     while node == i or node == n:

```

```

5     node = random.choice(n1)
6     g2 = graph.do(node)
7     action = lf[node]
8     #Remplacer par an external factor
9     s = f'Let\'s suppose that Damien\'s boss forced him to be {action}'
10    return s, g2

```

On obtient :

```

1 If someone is not wearing_color, then the probability that they are blue is
  0.5577032953033384
2 If someone is wearing_color, then the probability that they are not blue is
  0.44229670469666166
3 If someone is wearing_color, then the probability that they are red is
  0.5577032953033384
4 If someone is not wearing_color, then the probability that they are not white is
  0.44229670469666166
5 If someone is wearing_color, then the probability that they are not white is
  0.44229670469666166
6 There is a probability of 0.6381382257983802 that Neimad is not wearing_color today.
7 If someone is red, then the probability that they are not wearing_color is
  0.6381382257983802
8 There is a probability of 0.6381382257983802 that Neimad is not wearing_color today.
9 There is a probability of 0.5577032953033384 that Neimad is red today.
10 Let's suppose that Damien's boss forced him to be white
11 What is the probability that Neimad is red ?

```

5 Entraînement

5.1 Quelques notions

Maintenant que nous avons un jeu de données, il peut être intéressant d'essayer d'entraîner un LLM avec.

Définition 5.1 (LLM)

Les LLM (Large Language Models) sont des modèles d'apprentissage automatique capables de comprendre et de générer des textes en langage naturelle. Ils sont conçus pour être entraînés efficacement sur de très larges jeux de données. Un LLM est généralement constitué de plusieurs sous-modèles ayant chacun un rôle bien précis. Ils sont souvent entraînés sur des jeux de données très généraux puis chaque utilisateur modifie des parties spécifiques du réseau (fine-tuning) pour augmenter les performances du modèle sur des jeux de données particuliers.

Remarque 5.1. *Le mot comprendre est entre guillemets car les LLM, à proprement parler, ne comprennent pas ce qu'on leur dit et ne répondent pas à nos questions. Ils se contentent juste de générer la suite de mots la plus probable après l'entrée (question en générale) que l'on leur a donnée.*

Dans notre cas nous allons utiliser le modèle RoBERTa (A Robustly Optimized BERT Pretraining Approach).

Avant d'entraîner de d'entraîner notre modèle, il faut générer à l'aide de notre algorithme trois jeux de données.

- Un jeu d'entraînement. C'est le jeu de données qu'on va utiliser pour ajuster les poids du réseau.
- Un jeu de validation. Il est utilisé au cours de l'apprentissage pour adapter les valeurs des hyperparamètres (cf ci-dessous) et tester si le modèle n'est pas surentraîné.
- Un jeu de test. Il sert à tester les performances du modèle sur un jeu de données avec lequel le modèle n'a eu aucun contact.

Définition 5.2 (Hyperparamètres)

Paramètres qui régissent la manière dont le modèle s'adapte et évolue lors de l'entraînement. Par exemple la vitesse d'apprentissage est un hyperparamètre.

5.2 Algorithme d'entraînement

Pour entraîner ce modèle nous allons utiliser Python ainsi que les bibliothèques :

- *Panda* : pour utiliser notre jeu de données,
- *HuggingFace* : pour importer RoBERTa
- *TensorFlow* : pour entraîner notre modèle.

Voici les étapes importantes de l'algorithme d'entraînement :

1. Charger le drive et les données
2. Créer les deux datasets Train et Test
3. Charger le modèle
4. Charger le tokenizer qui va permettre de transformer les données au format compréhensible par le modèle
5. Transformer les données à l'aide du tokenizer
6. Entraîner notre modèle

```
1 drive.mount('/content/drive') #je vais charger les données depuis mon drive, permet de
   connecter mon drive
2
3 #on prend toujours la même seed pour qu'on puisse comparer plusieurs exécutions du
   code
4 seed = 0
5 validation_indices = random.Random(x=0).sample([i for i in range(100)], 20) #je n'ai
   qu'un seul jeu de données, je vais donc le couper en deux
6
7 out = pd.read_csv('/content/drive/MyDrive/regression_transformer/out.csv').reset_index
   (drop=True)
8
9 train_ds = out.iloc[~out.index.isin(validation_indices)]
10 test_ds = out.iloc[validation_indices] #on va prendre ici test_ds = validation_ds
11
12 #parametre du code, permet de faciliter la modification de celui-ci
13 batch_size = 8
14 learning_rate = 2e-5 #vitesse d'apprentissage, i.e. à quel point les poids sont modifi
   é. En générale on utilise la formule weight_{i+1} = weight_i - learning_rate*
   gradient(weight_i)
15 weight_decay = 0.01 #on veut entraîner notre modele plus vite au début car
   probabilistiquement, on est loin du minimum de la fonction d'erreur, vers la fin de
   l'entraînement c'est l'inverse
16 epochs = 5 #nombre de fois qu'on va parcourir le jeu de données pour entraîner notre
   modele
17 num_model = 1
18 max_length = 505 #longueur maximum d'une entrée (ici d'une énoncé), on veut max_lenght
   aussi faible que possible, quitte à supprimer quelques données qui ont une
   longueur anormalement grande
19 model_base = "/content/drive/MyDrive/regression_transformer/model_1/checkpoint-50"
20 device = "cuda" if torch.cuda.is_available() else "cpu"
21
22 #quelques fonction pour préparer les données pour qu'elles puissent être utilisées
   pour l'entraînement
23 def count_tokens(row, tokenizer):
24     input_ids = len(tokenizer(row['Simple']))['input_ids']
25     return input_ids
26
27 def tokenize(examples, tokenizer):
28     return tokenizer(examples["text"], truncation=True, padding='max_length')
29
30 #norme 2, c'est notre fonction d'erreur
31 def compute_metrics(eval_pred):
32     predictions, labels = eval_pred
33     rmse = mean_squared_error(labels, predictions, squared=False)
34     return {"rmse": rmse}
35
36 def preprocess(ds, tokenizer):
37     df_new = ds[['Simple', 'AnswerSimple']]
```

```

38 df_new.columns = ["text", "label"]
39 print(df_new.head())
40 dataset = Dataset.from_pandas(df_new)
41 return dataset.map(tokenize, fn_kwargs={"tokenizer": tokenizer}, batched=True)
42
43 #chargement du tokenizer, c'est lui qui se chargera de transformer les entrée du
44   langage naturelle vers un format comprehensible par le réseau
45 tokenizer = AutoTokenizer.from_pretrained(model_base, model_max_length=max_length)
46 #chargement du model
47 model = AutoModelForSequenceClassification.from_pretrained(model_base, num_labels = 1)
48   .to(device)
49
50 """## load pretrained model from checkpoint"""
51
52 train_ds['token_length'] = train_ds.apply(lambda x: count_tokens(x, tokenizer), axis =
53   1)
54 #train_ds['token_length'].describe()
55
56 train_ds['token_length'].value_counts()[:20].plot(kind='barh')
57
58 tokenized_dataset_train, tokenized_dataset_test = preprocess(train_ds.
59   reset_index(drop=True), tokenizer), preprocess(test_ds.reset_index(drop=True),
60   tokenizer)
61
62 args = TrainingArguments(
63   evaluation_strategy = "epoch",
64   save_strategy = "epoch",
65   learning_rate=learning_rate,
66   num_train_epochs=epochs,
67   report_to="none",
68   output_dir=f'/content/drive/MyDrive/regression_transformer/model_{num_model}',
69   weight_decay=weight_decay,
70   logging_steps = 1,
71   logging_dir="./log",
72   metric_for_best_model='rmse')
73
74 trainer = Trainer(
75   model,
76   args,
77   train_dataset= tokenized_dataset_train,
78   eval_dataset = tokenized_dataset_test,
79   tokenizer=tokenizer,
80   compute_metrics=compute_metrics
81 )
82
83 trainer.can_return_loss = True
84
85 trainer.train()

```

5.3 Résultats

Par faute de temps je n'ai pas eu le temps de faire des testes étendus sur le modèle. Il est toutefois possible de regarder ce qu'il donne dans des cas isolés.

```
[ ] test_ds['AnswerSimple'].iloc[0], test_ds['AnswerSimple'].iloc[1]
↔ (0.5577032953033384, 0.5577032953033384)

[ ] phrase = test_ds['Simple'].iloc[1]
    tokenized_sentence = tokenizer(phrase, return_tensors='pt').to("cpu")

▶ model.to("cpu")(**tokenized_sentence).logits[0][0].item()
↔ 0.5012671947479248
```

FIGURE 16 – Résultats

La première ligne donne le résultat que l'on souhaite avoir, la dernière celui qui est donné par le modèle.

On voit que l'on a un résultat proche.

De manière plus globale on obtient une erreur de 0,07 sur le jeu d'entraînement.

Epoch	Training Loss	Validation Loss	Rmse
1	0.004100	0.013554	0.116423
2	0.019300	0.010641	0.103156
3	0.016700	0.010625	0.103077
4	0.015100	0.009701	0.098492
5	0.009700	0.006274	0.079211

FIGURE 17 – Différentes erreurs au cours de la phase d'entraînement

6 Sources

- <https://medium.com/causality-in-data-science/confounding-colliding-d-separation-and-sleeping-wi-in-data-science/confounding-colliding-d-separation-and-sleeping-with-shoes-on-8ba43c976354>
- RuleTaker <https://arxiv.org/pdf/2002.05867>
- Cladder <https://arxiv.org/pdf/2312.04350>
- RoBERTa https://huggingface.co/docs/transformers/model_doc/roberta
- Problog https://dtai.cs.kuleuven.be/problog/natural_language/#example-1
- <https://en.wikipedia.org/wiki/Problog>
- #SAT <https://www.cs.cornell.edu/~sabhar/chapters/ModelCounting-SAT-Handbook-prelim.pdf>
- Fonctionnement Problog https://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=&ved=2ahUKEwjE89m7_pCIAxWufKQEHTFKE9YQFnoECBQQAQ&url=https%3A%2F%2Fliarias.kuleuven.be%2Fretrieve%2F112219&usg=A0vVaw36ShSe0hL_TT4mftkYNaHP&opi=89978449
- DO-operation : <https://arxiv.org/abs/2206.01802>

- Graphe Bayésien <https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/918>
- Sympy <https://www.sympy.org/en/index.html>
- Pycosat <https://pypi.org/project/pycosat/>
- Pgmpy <https://pgmpy.org/models/bayesiannetwork.html>
- Daft, tool used to render graphs <https://docs.daft-pgm.org/en/latest/>
- NetworkX <https://networkx.org>
- Tuto Panda <https://www.python-simple.com/python-pandas/panda-intro.php>
- Unigram, bibliothèque pour gérer des grammaires <https://github.com/sileod/unigram>
- Introduction to DO-calculus <https://plato.stanford.edu/entries/causal-models/#Coun>
- Introduction to Bayesian Network - Stanford <https://www.youtube.com/watch?v=U23yuPEACG0&t=2761s>
- Article on Bayesian Networks <https://arxiv.org/abs/1302.6811>

7 Annexe

Afin de ne pas rallonger inutilement le rapport, j'ai mis le code des deux algorithmes sur mon github : <https://github.com/Myskdias/ProbaPy/tree/main>